

Detalhamento Técnico

Sistema de Emissão de Notas Fiscais — Teste Técnico Korp ERP

Candidato: Luan Candido

1. Visão geral da solução

O sistema foi desenvolvido seguindo uma arquitetura de microsserviços, com um frontend em Angular consumindo duas APIs independentes em C# (.NET), responsáveis respectivamente pelo controle de estoque e pela emissão de notas fiscais.

Serviço	Porta / URL base
EstoqueService	http://localhost:5293
FaturamentoService	http://localhost:5000
Frontend Angular	http://localhost:4200

2. Frontend (Angular)

2.1 Ciclos de vida do Angular utilizados

Foi utilizado o ciclo de vida `ngOnInit` nos componentes `Produtos` e `Notas`. Esse hook é chamado automaticamente pelo Angular logo após a criação do componente, e foi usado para disparar o carregamento inicial dos dados (lista de produtos e lista de notas fiscais) assim que o usuário acessa cada tela, via chamada HTTP ao respectivo backend.

2.2 Uso da biblioteca RxJS

O RxJS é utilizado de forma indireta, através do `HttpClient` do Angular: toda chamada HTTP (GET, POST) retorna um `Observable`, que é consumido com o método `.subscribe()`, tratando separadamente os callbacks de sucesso (`next`) e erro (`error`). Essa abordagem foi usada em todas as operações assíncronas do sistema: carregar produtos, cadastrar produto, adicionar saldo, carregar notas, criar nota e imprimir nota.

2.3 Outras bibliotecas utilizadas

- `@angular/common/http` (`HttpClient`) — comunicação com as APIs REST dos dois microsserviços.
- `@angular/forms` (`FormsModule`) — utilizado para two-way data binding (`[(ngModel)]`) nos formulários de cadastro de produtos e emissão de notas fiscais.
- `@angular/router` (`RouterOutlet`, `RouterLink`) — roteamento entre as telas de `Produtos` e `Notas Fiscais`, e navegação via menu superior.
- `@angular/common` (`CommonModule`) — necessário para diretivas de template utilizadas nos componentes standalone.

2.4 Bibliotecas para componentes visuais

Não foi utilizada nenhuma biblioteca de componentes visuais de terceiros (como Angular Material, Bootstrap ou PrimeNG). Toda a estilização foi feita com CSS puro, escrito manualmente para cada componente, mantendo o projeto leve e sem dependências externas de UI.

3. Backend (C# / .NET)

3.1 Frameworks utilizados

- ASP.NET Core Web API — usado nos dois microsserviços (`EstoqueService` e `FaturamentoService`), no modelo de hospedagem mínimo (minimal hosting), com `Controllers` para organizar os endpoints REST.
- Entity Framework Core (EF Core) — ORM utilizado para mapear as classes de domínio (`Produto`, `NotaFiscal`, `ItemNota`) para tabelas relacionais e realizar as operações de persistência.

- Provider SQLite (Microsoft.EntityFrameworkCore.Sqlite) — cada microsserviço possui seu próprio banco de dados SQLite (estoque.db e faturamento.db), reforçando o isolamento entre os serviços.
- IHttpConnectionFactory — usado no FaturamentoService para realizar chamadas HTTP ao EstoqueService (comunicação entre microsserviços) durante a impressão de uma nota fiscal.

3.2 Gerenciamento de dependências

Não se aplica ao Golang, pois a implementação do backend foi feita em C#. O gerenciamento de pacotes foi realizado através do NuGet (dotnet add package), utilizado para instalar o Entity Framework Core, o provider de SQLite e as ferramentas de linha de comando (dotnet-ef) usadas para gerar e aplicar as migrations do banco de dados.

3.3 Tratamento de erros e exceções

O tratamento de erros foi feito de forma explícita em cada endpoint, retornando códigos HTTP apropriados de acordo com a situação:

- 400 (Bad Request) — usado para regras de negócio violadas, como tentar cadastrar um produto com código já existente, tentar imprimir uma nota fiscal que não está com status "Aberta", ou saldo insuficiente em estoque.
- 404 (Not Found) — usado quando um produto ou nota fiscal não é encontrado pelo identificador informado.
- 503 (Service Unavailable) — implementado especificamente para o cenário de falha de comunicação entre microsserviços: ao imprimir uma nota, o FaturamentoService chama o EstoqueService via HTTP; essa chamada é envolvida em um bloco try/catch que captura a exceção HttpRequestException lançada quando o serviço de destino está indisponível (por exemplo, se o EstoqueService for derrubado). Nesse caso, o sistema retorna uma mensagem amigável ao usuário ("O serviço de Estoque está indisponível no momento. Tente novamente em instantes.") em vez de expor o erro técnico bruto, e a nota permanece com status "Aberta", podendo ser impressa novamente assim que o serviço de Estoque voltar a responder — comportamento validado manualmente derrubando o EstoqueService e confirmando a recuperação automática do fluxo.

3.4 Uso de LINQ

Sim, LINQ foi utilizado extensivamente em conjunto com o Entity Framework Core para consultas ao banco de dados, sempre em suas versões assíncronas:

- FirstOrDefaultAsync(p => p.Codigo == codigo) — para buscar um produto específico pelo código.
- AnyAsync(p => p.Codigo == novoProduto.Codigo) — para verificar se já existe um produto cadastrado com determinado código antes de permitir um novo cadastro.
- ToListAsync() — para obter a lista completa de produtos ou notas fiscais.
- Include(n => n.Itens) — para realizar o carregamento relacionado (eager loading) dos itens de cada nota fiscal junto com a nota principal, já que se trata de tabelas separadas relacionadas por chave estrangeira.

4. Arquitetura de microsserviços

O sistema foi estruturado em dois serviços independentes, cada um com sua própria base de código, banco de dados e ciclo de execução:

- EstoqueService — responsável pelo cadastro de produtos e controle de saldo em estoque.
- FaturamentoService — responsável pela emissão e impressão de notas fiscais.

A comunicação entre os serviços ocorre de forma síncrona via HTTP: ao imprimir uma nota fiscal, o FaturamentoService realiza uma chamada REST ao EstoqueService para dar baixa no saldo de cada produto envolvido na nota. Essa separação permite que cada serviço evolua e seja implantado de forma independente.

5. Persistência de dados

Ambos os microsserviços utilizam Entity Framework Core com SQLite para persistência real em disco, substituindo uma abordagem inicial de listas em memória. Foram geradas migrations (dotnet ef migrations add)

para criar o esquema do banco a partir das classes de domínio, e a persistência foi validada reiniciando os serviços e confirmando que os dados cadastrados permanecem disponíveis.